

TABLE OF CONTENTS

Flutter App Architecture with Riverpod: An Introduction	1
Riverpod is not very opinionated	2
Riverpod + Reference App Architecture = 🍌	2
The Presentation Layer	4
The Domain Layer	4
The Data Layer	5
The Application Layer	6
Wrap Up	7
Where to go from here?	8
Flutter Foundations Course Now Available	9
Flutter Foundations Course	9
Want More?	9
Flutter In Production	10
Flutter & Firebase Masterclass	10
The Complete Dart Developer Guide	10
Flutter Animations Masterclass	10



Flutter App Architecture with Riverpod: An Introduction

#dart

#flutter

#riverpod

#app-architecture

#design-patterns



Andrea Bizzotto

Updated Sep 1, 2023 · 8 min read

When building complex apps, choosing the correct **app architecture** is crucial, as it allows you to **structure** your code and support your codebase as it grows.

Good architecture should help you **handle complexity without getting in the way**. But it's not easy to get it right:

- “Not enough” architecture leads to poorly organized code that lacks clear conventions
- “Too much” of it leads to over-engineering, making it hard to make even simple changes

In practice, things can be quite nuanced, and it can be tricky to get the right balance.

So in this article, I'm sharing a new reference architecture that you can use to:

- **ensure a good separation of concerns** between your **UI code** (1), your **business logic** (2), and your **data access logic** (3)
- **easily fetch and cache data** with minimal boilerplate code
- **perform data mutations** while handling the UI states (data, loading, error) in a predictable way
- **write testable code** and mock dependencies in a breeze

This architecture leans heavily on the Riverpod package, making the most of its **reactive caching** and **data-binding** features.

But why do we need a reference architecture in the first place?

Riverpod is not very opinionated

Riverpod is great. I use it in all my apps and have written many tutorials about it - but the official docs don't offer any guidance about how to structure your code.

As a result, you're on your own when it comes to making important decisions such as:

- How to separate your UI code from your business logic and the data access logic?
- Which classes should you write? What are their responsibilities? And how do they communicate with each other?
- How do you make your code more scalable - so that different team members can work on multiple features independently?
- How should you organize your files?
- What about error handling? Where should it take place, and how should errors propagate to the UI?
- Which providers should you use, and where should they be declared?

These decisions matter. And if you leave them to chance, you'll end up with performance problems, bugs, and ongoing maintenance issues that will affect your development speed.

And this is where having a very opinionated app architecture makes all the difference. 📌

Riverpod + Reference App Architecture = 🤞

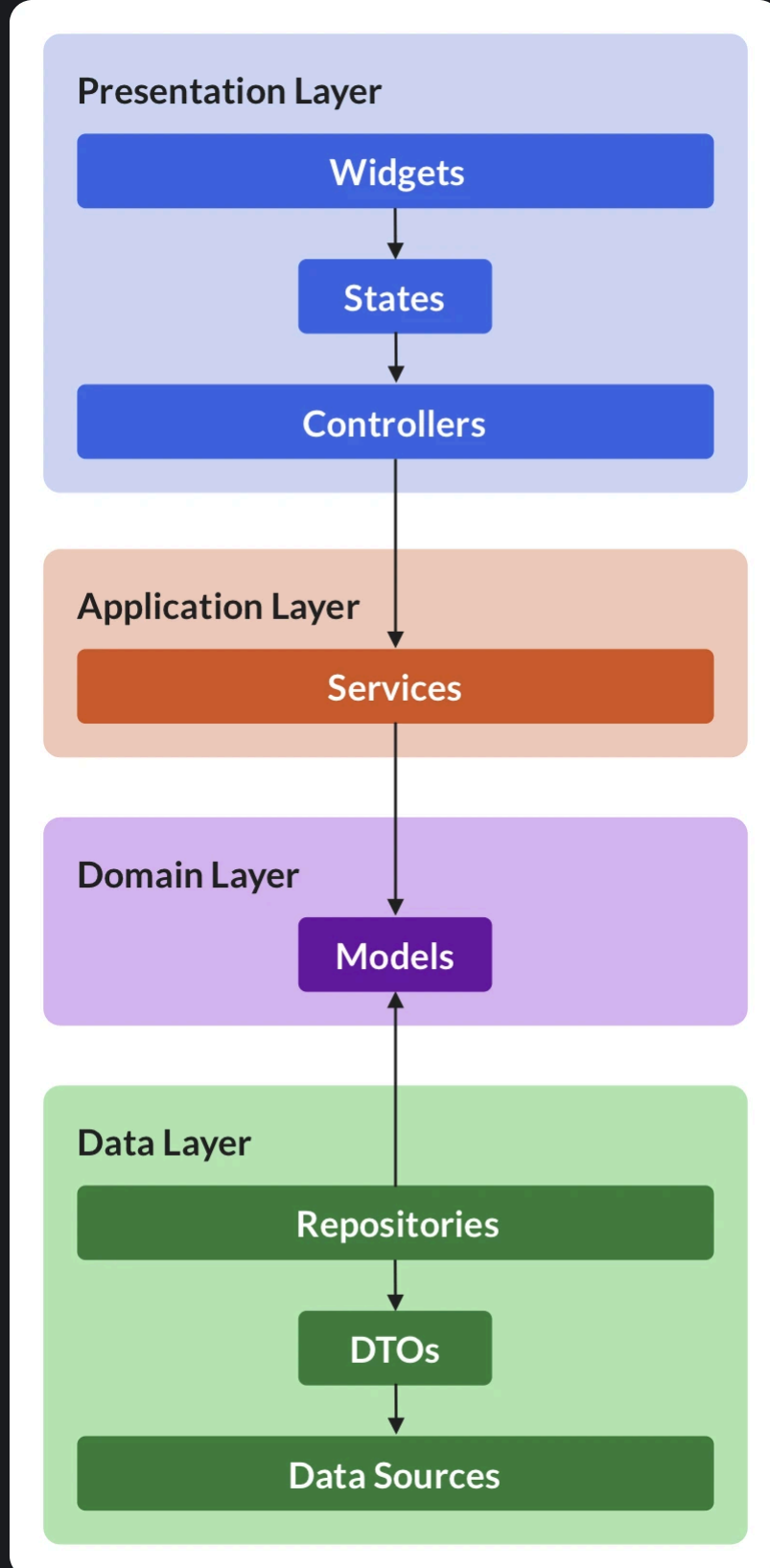
While building Flutter apps of varying complexity, I've experimented a lot with app architecture and developed a deep understanding of what works well and what doesn't.

The result is a **reference architecture** that I've used in all my latest projects.

For lack of a better name, I'll call this Riverpod Architecture - though keep in mind that this is just my take on it and not an "official" architecture endorsed by Remi Rousselet (the author of Riverpod).

This architecture comprises four layers (**data**, **domain**, **application**, and **presentation**).

Here's a preview:



App architecture using data, domain, application, and presentation layers. Arrows show the dependencies between layers.

Each of these layers has its own responsibility, and there's a **clear contract** for how communication happens across boundaries.

So let's take a closer look at each layer. 📌

Since there is much to cover, this article only includes a conceptual, high-level overview of the four layers. Below you'll find links to separate articles covering each layer in more

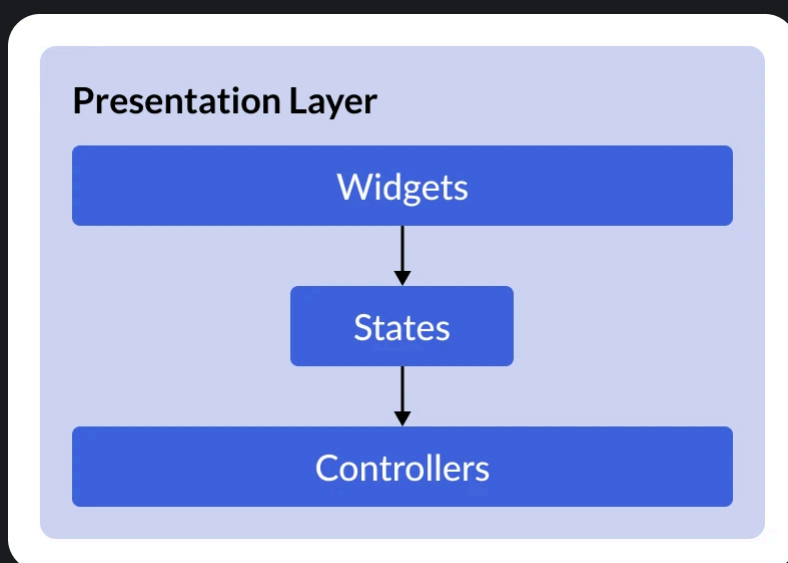
The Presentation Layer

This is often called the UI layer. And [this guide about Android app architecture](#) describes it well:

The role of the UI is to display the application data on the screen and also to serve as the primary point of user interaction. Whenever the data changes, either due to user interaction (like pressing a button) or external input (like a network response), the UI should update to reflect those changes. *Effectively, the UI is a visual representation of the application state as retrieved from the data layer.*

In our architecture, the presentation layer contains two main types of components:

- **Widgets**, which are a representation of the data to be displayed on screen.
- **Controllers**, which perform asynchronous data mutations and manage the widget state.



Flutter App Architecture: the presentation layer. Arrows show the dependencies between layers.

The controllers themselves are usually represented as `AsyncNotifier` subclasses, and I've covered them in detail in this separate article:

- [Flutter App Architecture: The Presentation Layer](#)

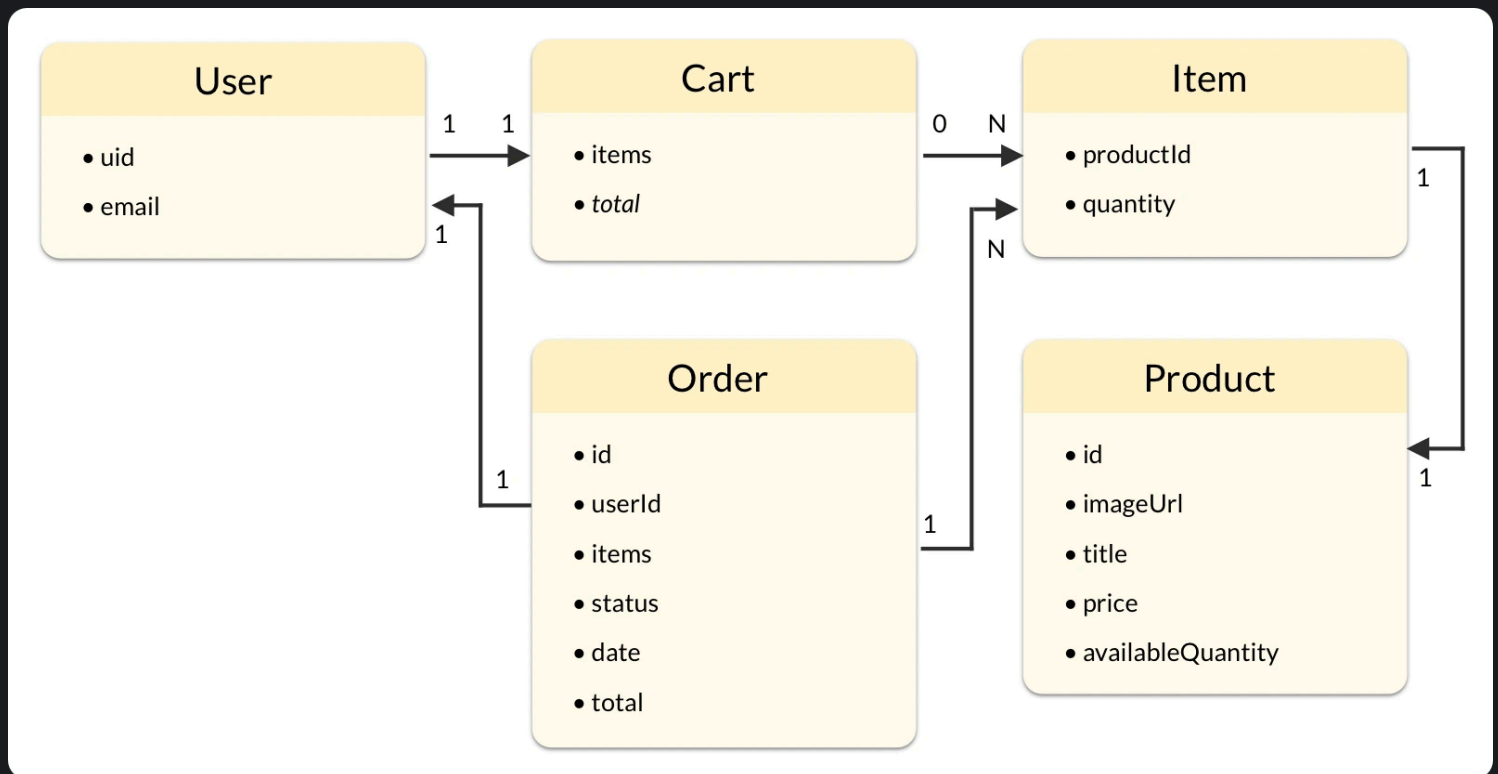
The Domain Layer

The primary role of the domain layer is to define **application-specific model classes** that represent the data that comes from the data layer.

Model classes are simple data classes and have the following requirements:

- They are always immutable.
- They contain serialization logic (such as `fromJson` and `toJson` methods).
- They implement the `==` operator and the `hashCode` method.

Using an eCommerce application as an example, we could identify the following entities and represent them as model classes:



eCommerce app: entities and their relationships

Model classes may depend on other model classes (e.g. a `ShoppingCart` class may contain a list of `Products`). But they don't know where to get the data from and have no other dependencies. As such, model classes can be imported and used everywhere else in your app (widgets, controllers, services, repositories).

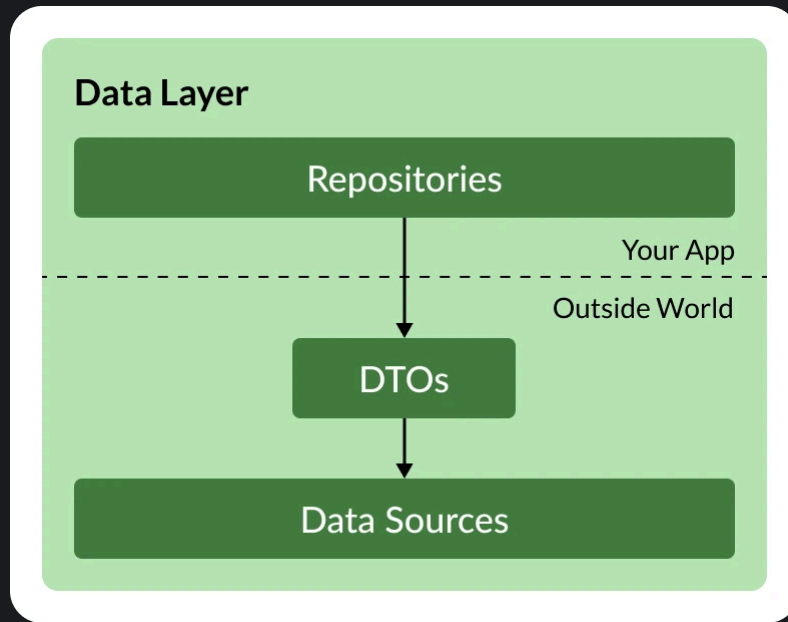
To more easily define the properties and methods in your data classes, you can use packages such as [Freezed](#) and [Equatable](#), or tools such as the [Dart Data Class Generator](#).

To learn more about the domain layer and model classes, read this:

- [Flutter App Architecture: The Domain Model](#)

The data layer contains three types of classes:

- **Data Sources**, which are 3rd party APIs used to communicate with the outside world (e.g. a remote database, a REST API client, a push notification system, a Bluetooth interface).
- **Data Transfer Objects** (or DTOs), which are returned by the data sources. DTOs are often represented as unstructured data (such as JSON) when sending data over the network
- **Repositories**, which are used to access DTOs from various sources, such as a backend API, and make them available as type-safe **model classes** (a.k.a. entities) to the rest of the app.



Flutter App Architecture: the data layer. Arrows show the dependencies between layers.

Note that both data sources and DTOs are **external dependencies**. To use them, you import packages into your app and **consume** their APIs.

On the other hand, repositories are classes that live in your codebase, so it's your job to implement them and **design** their API.

If your Flutter app talks to a local or remote database, that database is the single source of truth for your data. As far as your app is concerned, **repositories are the gateway** to that source of truth. This app architecture accounts for this by implementing an **unidirectional data flow** from the data layer all the way into the UI.

To learn more about repositories and the data layer, read this:

- [Flutter App Architecture: The Repository Pattern](#)

The Application Layer

When building complex apps, we may find ourselves writing logic that:

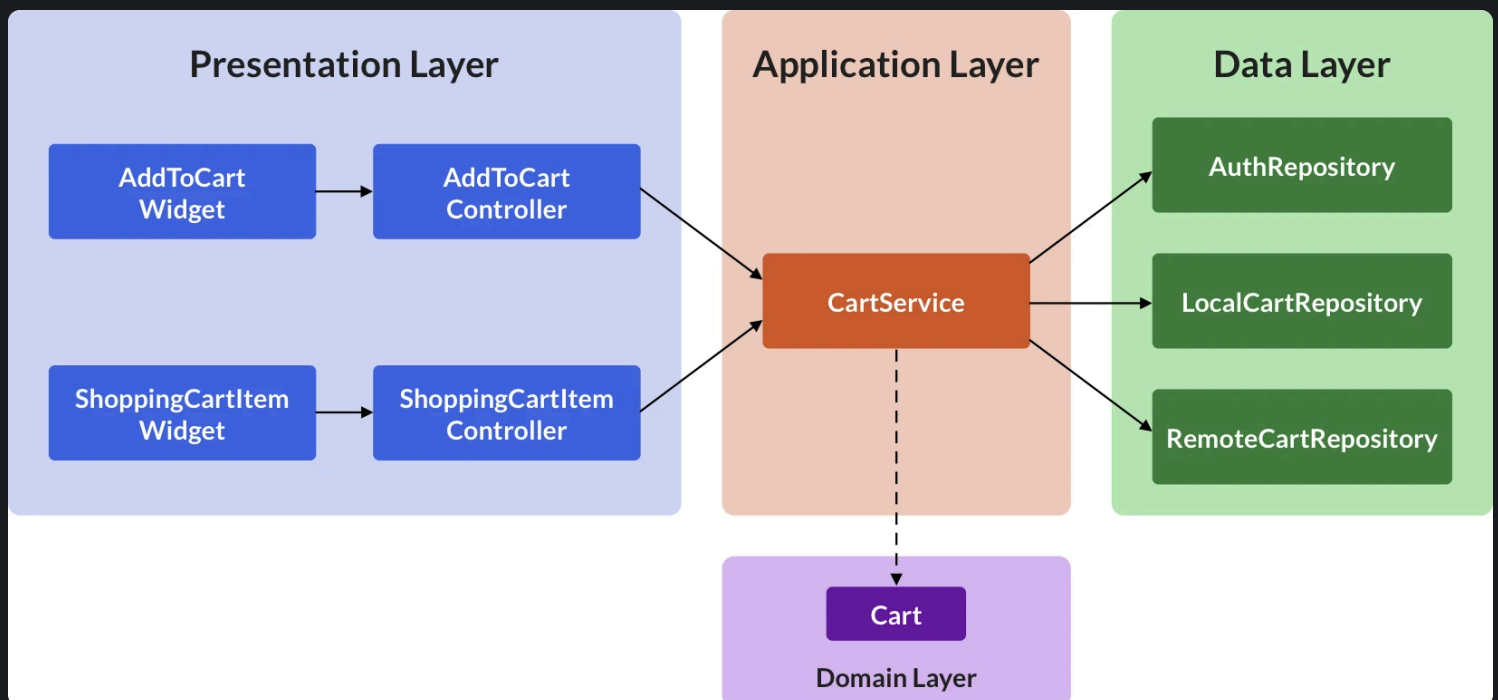
- depends on multiple data sources or repositories
- needs to be used (shared) by more than one widget

In this case, it's tempting to put that logic inside the classes we already have (controllers or repositories).

But this leads to poor separation of concerns, making our code harder to read, maintain, and test.

To address this, we can introduce a new, **optional** layer called the application layer. Inside it, we can add service classes, which act as the middle-man between the controllers (which only manage the widget state) and the repositories (which talk to different data sources).

Here's an example showing a `CartService` class that mediates between controllers and repositories:



A shopping cart example feature showing how to use a service class to mediate between controllers and repositories

To learn more about the application layer, read this:

- [Flutter App Architecture: The Application Layer](#)

Wrap Up

So far, we've learned about the four main layers in my Riverpod architecture and the classes inside them (widgets, controllers, models, services, repositories, data sources).

But how do all these different classes interact with each other, and how can we use them to build working features in our apps?

This is where Riverpod and all its useful providers come in. And when building mobile apps, most of the work we do boils down to two things:

- Fetch data and show it in the UI
- Perform data mutations in response to input events

To learn more about this, you can read this follow-up article:

- [How to Fetch Data and Perform Data Mutations with the Riverpod Architecture](#)

Where to go from here?

If you need help choosing the most appropriate project structure for your Flutter apps, I've got you covered:

- [Flutter Project Structure: Feature-first or Layer-first?](#)

If you want to explore other popular architectures (such as MVP, MVVM, or Clean Architecture) and understand how they compare with the architecture proposed here, read this:

- [A Comparison of Popular Flutter App Architectures](#)

To learn more about each of the four layers in the Riverpod architecture, read the other articles in this series:

- [Flutter App Architecture: The Repository Pattern](#)
- [Flutter App Architecture: The Domain Model](#)
- [Flutter App Architecture: The Presentation Layer](#)
- [Flutter App Architecture: The Application Layer](#)

And if you want to go deeper still, you can check out my Flutter Foundations course, where I guide you through the implementation of a complex eCommerce app using the app architecture

we've covered here, along with many useful techniques you won't find elsewhere. 📌

Flutter Foundations Course Now Available

I launched a brand new course that covers Flutter app architecture in great depth, along with other important topics like state management, navigation & routing, testing, and much more:



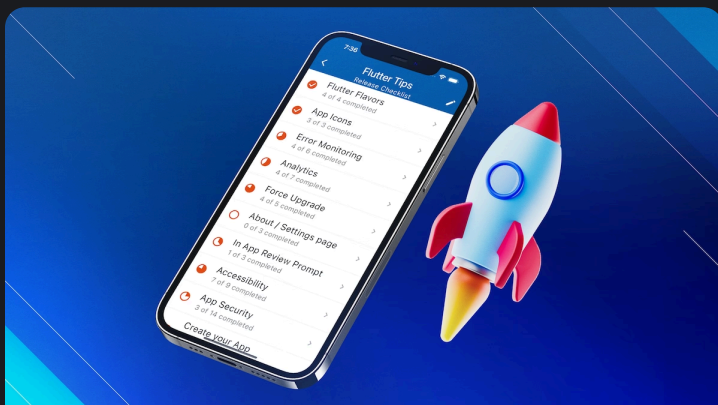
Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

Invest in yourself with my high-quality Flutter courses.



Flutter In Production



INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀

Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.



CODE WITH ANDREA

Copyright © 2020-present Coding With Flutter Limited

[Contact](#)

[Twitter](#)

[Discord](#)

[GitHub](#)

[RSS](#)

[Meta](#)

[Privacy Policy](#)

[Terms of Use](#)